

Functions in Python

What are functions?

A function is a reusable block of code designed to perform a specific task. You define the logic once and call it whenever you need that task.

What is their role?

Functions make programs organized, reusable, readable, testable, and easier to maintain. They let you break a large problem into smaller, understandable pieces.

DEFINE → write the behavior

CALL → execute it

INPUT → give data

OUTPUT → receive a result

01 — WHY FUNCTIONS EXIST

If the same logic is needed many times, copying it creates duplication. A function lets you write the logic once and reuse it.

```
● ● ● python

def calculate_total(price, quantity):
    return price * quantity

print(calculate_total(100, 3))
print(calculate_total(250, 2))

OUTPUT

300
500
```

The function name describes a reusable behavior. This makes the program easier to read and easier to change.

02 — DEF, NAME, PARENTHESES & INDENTATION

Python uses **def** to define a function. The name identifies it; parentheses hold parameters; the colon starts the body; indentation defines which statements belong to it.

```
● ● ● python

def greet():
    print("Hello!")

greet()

OUTPUT

Hello!
```

Defining a function normally does not execute its body. The body runs when you call the function.

03 — CALLING A FUNCTION

A function call is the moment Python executes the function body.

```
● ● ● python

def show_message():
    print("Python is fun!")

show_message()
show_message()

OUTPUT

Python is fun!
Python is fun!
```

Think: definition = what the function does; call = when it does it.

04 — PARAMETERS & ARGUMENTS

A **parameter** is a variable in the function definition. An **argument** is the actual value supplied when calling the function.

● ● ● python

```
def greet(name):  
    print("Hello", name)  
  
greet("Hammad")  
greet("Ali")
```

OUTPUT

```
Hello Hammad  
Hello Ali
```

name is the parameter. Hammad and Ali are arguments.

05 — RETURN

return sends a value back to the caller. It is different from `print()`: `print` displays; `return` provides a value the rest of the program can use.

● ● ● python

```
def add(a, b):  
    return a + b  
  
result = add(10, 5)  
print(result)
```

OUTPUT

```
15
```

● ● ● python

```
def square(x):  
    return x * x  
  
x = square(5)  
y = square(x)  
print(y)
```

OUTPUT

```
625
```

06 — RETURN STOPS THE FUNCTION

When `return` executes, the current function call ends. Code after that `return` in the same execution is not reached.

● ● ● python

```
def test():  
    print("Start")  
    return 10  
    print("Never runs")  
  
print(test())
```

OUTPUT

```
Start  
10
```

07 — MULTIPLE RETURN VALUES

Python can return several values. They are packaged as a tuple and can be unpacked into separate variables.

● ● ● python

```
def calculate(a, b):  
    return a + b, a - b  
  
x, y = calculate(10, 4)  
print(x)  
print(y)
```

OUTPUT

```
14  
6
```

08 — DEFAULT PARAMETERS

A default parameter is used when the caller does not provide that argument.

● ● ● python

```
def greet(name, message="Hello"):  
    print(message, name)  
  
greet("Hammad")  
greet("Hammad", "Welcome")
```

OUTPUT

```
Hello Hammad  
Welcome Hammad
```

Default parameters make a function flexible while keeping common calls short.

09 — POSITIONAL & KEYWORD ARGUMENTS

Positional arguments match parameters by order. Keyword arguments match them by parameter name.

● ● ● python

```
def student(name, age, field):  
    print(name, age, field)  
  
student("Hammad", 18, "Data Science")  
student(age=18, field="Data Science", name="Hammad")
```

OUTPUT

```
Hammad 18 Data Science  
Hammad 18 Data Science
```

10 — SCOPE: LOCAL & GLOBAL

Scope means where a name can be accessed. Variables created inside a function normally belong to that function's local scope.

● ● ● python

```
def show():  
    message = "Inside"  
    print(message)  
  
show()
```

OUTPUT

Inside

Prefer passing data into functions and returning results instead of relying heavily on global state. This makes code easier to test and reason about.

11 — FUNCTIONS & LISTS / MUTABLE DATA

When a list is passed to a function, the function can mutate that same list. Changes such as `append()` can therefore be visible to the caller.

● ● ● python

```
def add_item(items):  
    items.append("Python")  
  
languages = ["C", "Java"]  
add_item(languages)  
print(languages)
```

OUTPUT

['C', 'Java', 'Python']

This becomes important when working with collections and datasets.

12 — *ARGS

***args** lets a function accept any number of positional arguments. Inside the function, `args` is a tuple.

● ● ● python

```
def total(*numbers):  
    result = 0  
    for number in numbers:  
        result += number  
    return result  
  
print(total(10, 20))  
print(total(1, 2, 3, 4))
```

OUTPUT

30
10

The star is the important syntax; `args` is only the conventional name.

13 — **KWARGS

****kwargs** lets a function accept any number of keyword arguments. Inside the function, `kwargs` behaves like a dictionary.

● ● ● python

```
def show_profile(**info):
    for key, value in info.items():
        print(key, "=", value)

show_profile(name="Hammad", field="Data Science")
```

OUTPUT

```
name = Hammad
field = Data Science
```

14 — PARAMETER ORDER

As function signatures become advanced, order matters. A useful model is: regular parameters → defaults → *args → keyword-only parameters → **kwargs.

● ● ● python

```
def example(name, age=18, *args, active=True, **kwargs):
    print(name, age, args, active, kwargs)

example("Hammad", 18, "DS", "Python", active=True, city="Gilgit")
```

OUTPUT

```
Hammad 18 ('DS', 'Python') True {'city': 'Gilgit'}
```

Do not memorize this until normal parameters, return, defaults, and keyword arguments are comfortable.

15 — FUNCTIONS ARE OBJECTS

Python functions are first-class objects. They can be stored in variables, passed into other functions, and returned from functions.

● ● ● python

```
def greet():
    return "Hello"

message_function = greet
print(message_function())
```

OUTPUT

```
Hello
```

No parentheses in the assignment: greet refers to the function; greet() calls it.

16 — HIGHER-ORDER FUNCTIONS

A higher-order function accepts a function, returns a function, or both.

● ● ● python

```
def apply_twice(function, value):  
    return function(function(value))  
  
def double(x):  
    return x * 2  
  
print(apply_twice(double, 3))
```

OUTPUT

12

This idea leads naturally to callbacks, `map()`, `filter()`, decorators, and functional programming patterns.

17 — LAMBDA FUNCTIONS

A **lambda** is a small anonymous function written as an expression.

● ● ● python

```
square = lambda x: x * x  
print(square(5))
```

OUTPUT

25

Use lambda for genuinely small operations. A normal `def` is often clearer for reusable or complex logic.

18 — MAP() & FILTER()

map() transforms each item. **filter()** keeps items for which the function returns True.

● ● ● python

```
numbers = [1, 2, 3, 4]  
squared = list(map(lambda x: x * x, numbers))  
print(squared)
```

OUTPUT

[1, 4, 9, 16]

● ● ● python

```
numbers = [1, 2, 3, 4, 5, 6]  
evens = list(filter(lambda x: x % 2 == 0, numbers))  
print(evens)
```

OUTPUT

[2, 4, 6]

19 — RECURSION

Recursion means a function calls itself. It needs a **base case** that stops the recursion and a recursive step that moves toward that case.

● ● ● python

```
def countdown(n):
    if n == 0:
        print("Done")
        return
    print(n)
    countdown(n - 1)

countdown(3)
```

OUTPUT

```
3
2
1
Done
```

Recursion is useful for trees, divide-and-conquer algorithms, and some DSA problems. For simple repetition, loops are often clearer.

20 — DOCSTRINGS

A docstring is a string immediately inside a function definition that documents its purpose.

● ● ● python

```
def calculate_area(width, height):
    """Return rectangle area."""
    return width * height

print(calculate_area(5, 4))
```

OUTPUT

```
20
```

21 — TYPE HINTS

Type hints communicate expected parameter and return types. Python generally does not enforce them automatically at runtime.

● ● ● python

```
def add(a: int, b: int) -> int:
    return a + b

print(add(10, 5))
```

OUTPUT

```
15
```

Type hints become especially useful in larger projects, IDEs, linters, and type-checking workflows.

22 — DESIGNING GOOD FUNCTIONS

Ask five questions: What exact task does it perform? What inputs does it need? What should it return? What invalid cases exist? Can another part of the program reuse it?

● ● ● python

```
def calculate_discount(price, percent):  
    discount = price * (percent / 100)  
    return price - discount  
  
print(calculate_discount(1000, 10))
```

OUTPUT

900.0

Good functions usually have a clear responsibility, clear inputs, and a predictable output.

23 — REAL-WORLD FUNCTION STRUCTURE

Functions let you split a larger application into meaningful operations instead of creating one giant script.

● ● ● python

```
def add_expense(expenses, amount):  
    expenses.append(amount)  
  
def calculate_total(expenses):  
    return sum(expenses)  
  
expenses = []  
add_expense(expenses, 500)  
add_expense(expenses, 1200)  
print(calculate_total(expenses))
```

OUTPUT

1700

This same structure appears in mobile apps, backend services, automation, APIs, and data projects.

24 — FUNCTIONS IN DATA SCIENCE & ML

Functions are everywhere in Data Science: cleaning data, transforming features, calculating metrics, validating records, evaluating models, and preparing reusable pipelines.

● ● ● python

```
def normalize(value, minimum, maximum):  
    return (value - minimum) / (maximum - minimum)  
  
print(normalize(75, 0, 100))
```

OUTPUT

0.75

● ● ● python

```
def accuracy(correct, total):  
    if total == 0:  
        return 0  
    return correct / total  
  
print(accuracy(92, 100))
```

OUTPUT

0.92

Later, pandas, NumPy, and scikit-learn provide many ready-made functions. Understanding inputs, outputs, parameters, and return values makes those libraries much easier to learn.

25 — PRACTICE LADDER

Solve these without looking up the answer. Define the function, call it, predict the result, then test it.

LEVEL 1 — BASIC

Greeting, addition, subtraction, multiplication, even/odd.

LEVEL 2 — PARAMETERS

Rectangle area, Celsius to Fahrenheit, maximum of two values, percentage.

LEVEL 3 — RETURN

Factorial, square, cube, digit sum, prime test.

LEVEL 4 — DEFAULTS

Profile function with default age; greeting with default message.

LEVEL 5 — COLLECTIONS

List total, average, largest value, count of evens, filtered values.

LEVEL 6 — *ARGS / **KWARGS

Total any number of values; print arbitrary profile fields.

LEVEL 7 — RECURSION

Countdown, factorial, Fibonacci.

LEVEL 8 — MINI PROJECT

Build a finance utility module: add expense, total, largest expense, filter expenses, budget percentage.

26 — QUIZ: PREDICT BEFORE YOU RUN

First write the output or result on paper. Then run the code and explain why.

Q1 What happens?

● ● ● python

```
def add(a, b):  
    return a + b  
  
print(add(4, 6))
```

Q2 What happens?

● ● ● python

```
def greet(name):  
    print(name)  
  
greet("Hammad")
```

Q3 What happens?

● ● ● python

```
def test():  
    print("A")  
    return 10  
    print("B")  
  
print(test())
```

Q4 What happens?

● ● ● python

```
def square(x):  
    return x * x  
  
result = square(7)  
print(result)
```

Q5 What happens?

● ● ● python

```
def show(*args):  
    print(args)  
  
show(1, 2, 3)
```

Q6 What happens?

● ● ● python

```
def show(**kwargs):  
    print(kwargs)  
  
show(name="Hammad", field="DS")
```

Q7 What happens?

● ● ● python

```
def double(x):  
    return x * 2  
  
def apply_twice(fn, value):  
    return fn(fn(value))  
  
print(apply_twice(double, 3))
```

Q8 What happens?

● ● ● python

```
def make_message(name="User"):  
    return "Hello " + name  
  
print(make_message())  
print(make_message("Hammad"))
```

27 — FUNCTION CHALLENGES

1. Create `is_prime(n)`, then use a loop to print primes from 1–100.
2. Build an expense analyzer with `total()`, `average()`, `maximum()`, and `filter_expenses()`.
3. Build a student analyzer returning average, highest, lowest, and pass count.
4. Build a calculator using separate functions for `+`, `-`, `×`, `÷`.
5. Create `normalize()`, `accuracy()`, and `error_rate()` utilities for sample ML data.
6. Write a higher-order function that accepts another function and applies it to every list item.

28 — FUNCTIONS CHEAT SHEET

● ● ● python

```
# Define and call
def greet():
    print("Hello")
greet()

# Parameters + return
def add(a, b):
    return a + b

# Default parameter
def greet(name="User"):
    return "Hello " + name

# Keyword argument
greet(name="Hammad")

# *args
def total(*numbers):
    return sum(numbers)

# **kwargs
def profile(**info):
    return info

# Function as a value
fn = greet

# Pass a function
def apply(fn, value):
    return fn(value)

# Lambda
square = lambda x: x * x

# Recursion
def countdown(n):
    if n == 0:
        return
    print(n)
    countdown(n - 1)
```

FUNCTION = reusable behavior • **PARAMETER** = input name • **ARGUMENT** = actual value • **RETURN** = send result back • ***args** = many positional arguments • ****kwargs** = many keyword arguments

The core skill is to look at a problem and ask: **What task can I isolate? What inputs does it need? What result should it return?**

Next path: modules & packages → exceptions → file handling → OOP → data structures → algorithms / DSA → libraries and frameworks.